CSC3065 Cloud Computing
Assessment 3: Search Engine
Submission

Student Number: 40073762

Name: Maksimas Lajauskas

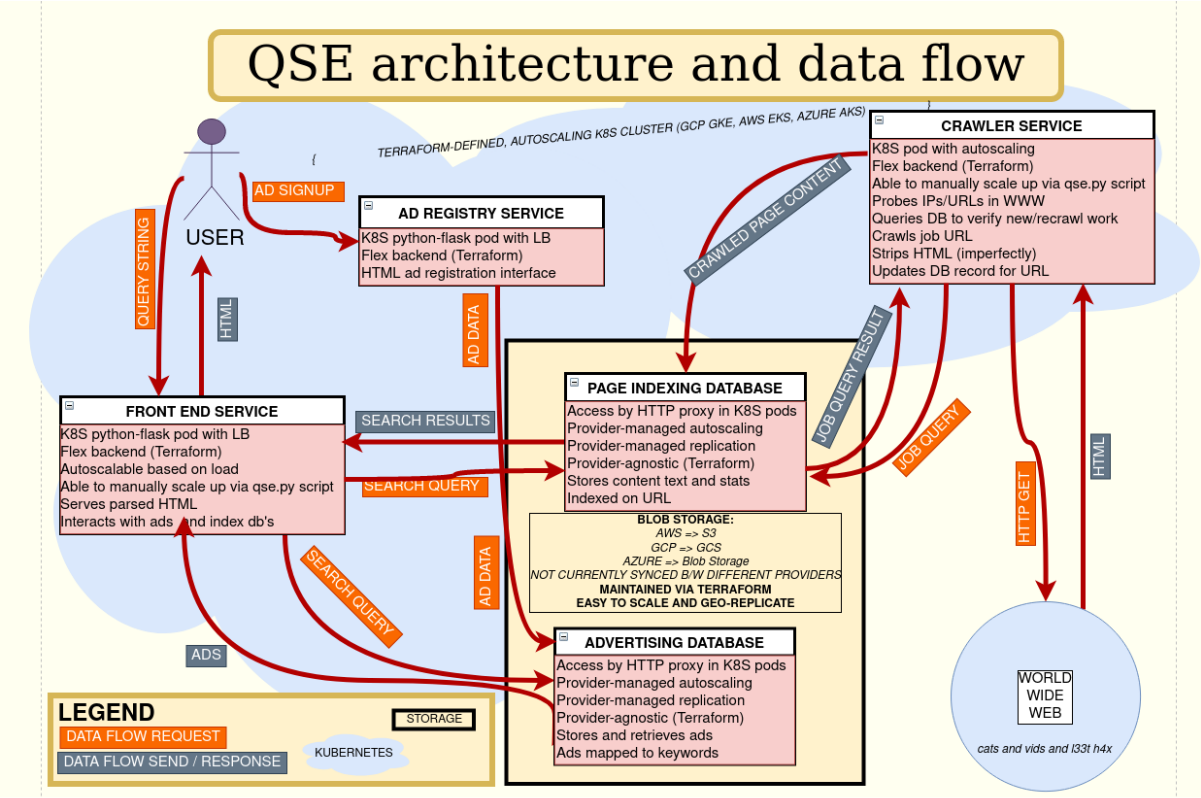
Submission Checklist

Make sure you have included the following:

- Report (this document) including all repository links
- https://github.com/maksimas-lajauskas/csc3065a3/
- Video of implementation working
- Source code fully included in the zip file

Architectural Design

Here you should detail a justified design for a scalable architecture of a search engine.



I based the implementation on Kubernetes and Terraform due to both of the technologies' flexibility to run on any provider currently supported by Terraform which currently includes most big CSPs including Yandex, Alibaba, and Tencent clouds in addition to the AWS, GCP and Azure based implementations I made for this submission.
The crawler service takes information from storage using ticket-blobs which are shared by all crawlers and deleted on pickup (with the option to run in "discovery mode" picking random ips and attempting to parse any webpage offerings).
The ads service is a simple web form for registering adverts with QSE that get returned with matching keyword searches in the frontend service.
The frontend service provides a web interface and uses python to query index and ads storage to serve dynamic pages from flask templates.
Currently there is no synchronisation between different providers, however the code has been left extensible to add a rest api capability to share and allocate crawl tickets across providers.

Implementation

Include here the implementation details of each of your components. Include diagrams, screenshots, or any additional details (justifications, decisions, testing, outputs, pipelines, etc) as you need to make clear how you have used "industry-ready approaches and techniques".

Each provider definition for the search engine consists of a main Terraform file containing core components like the kubernetes cluster and storage with mappings to variables for frequently configurable values like initial deployment region and pod replica scaling limits leaving more complex functionality to require modifying the Terraform definitions e.g. changing to a different storage type.

The qse modules are defined dynamically at deployment time through a script that packages credentials into the pods' environment variables. All pods are defined as kubernetes deployments in Terraform and the pod images are built to be provider agnostic with a simple interface mesh to each provider's cloud storage API that is included with each image, only needing environment variable configuration to function on a given provider. This separates the logic from the implementation further allowing for adding more storage interfaces in the future should the requirements change e.g. to include Azure CosmosDB or GCP BigTable.

The current blob storage offerings were chosen for their flexibility should further extensions like video files become required as well as for the capability to configure horizontal scaling and replication via altering the Terraform definitions.

Deployment Scripting and Management

All the detail of any deployment and management approaches (techniques, scripts, etc) you have used.

The deployment is managed from a python script as part of the repository that takes CLI config arguments and gives the option to update the terraform variable and pod definitions from command line options and pulls-pushes images from a docker hub repository to verify the newest image is being used and gives the option to configure kubectl to access the cluster.

Monitoring and Reporting

How have you setup monitoring and reporting? Ideas to put here include: What do you monitor (why?) and how does alerting/notifications work? What type of reports (examples)?